

Fix: Empty footer links, and the cast that hid them

Type: Fix

Status: verified

Branch: `fix/empty-footer-links-and-the-cast-that-hid-them`

The problem

Three links in the footer's Services column render with no text. Live now, on both locales, as served by production:

```
href=/nails    text=""
href=/lashes   text=""
href=/makeup   text=""
href=/gallery  text="The gallery"
```

A link with no accessible name is an accessibility defect as well as a visible gap: a screen reader announces "link" and nothing else.

Cause

`SiteFooter` resolves labels against `t.footer`, but `nails`, `lashes` and `makeup` exist only in `t.nav`. `gallery` renders because it happens to exist in both. The lookup returns `undefined` and React renders nothing.

Why the build did not catch it

`components/shell/SiteFooter.tsx:31`:

```
const label = (key: string) => t.footer[key as keyof Dictionary["footer"]];
```

The `as keyof` cast tells TypeScript the string is a valid key. It silences exactly the error that would have caught this.

Feature 2 built a build-time guarantee that a missing translation fails the build, and this cast walks around it three files later. The same cast sits in `SiteHeader.tsx:50` and `MobileNav.tsx:124` for `t.nav`; those are correct today by luck, not by checking.

Why verification missed it

The feature 2 checks grepped for Spanish words being **present**. Nothing checked for a rendered string being **empty**. "Uñas appears on `/es`" says nothing about three blanks elsewhere on the page.

The fix

Two parts, and the second matters more than the first.

1. **Make the links work.** Give the footer's service entries labels that resolve. Prefer reusing the existing `nav` labels over duplicating the same four words into `footer`: a second copy is a second thing to keep in sync, and the words are identical. 2. **Delete the three `as keyof` casts** so the lookups are type-checked. Type `NavItem.key` as a union drawn from the dictionary rather than `string`, so a key with no matching entry becomes a compile error.

Part 1 alone leaves the trap armed for whoever adds the next nav item.

Must not break:

- The missing-Spanish-string build error from feature 2. This should strengthen that guarantee, never weaken it.
- Existing labels. `The gallery`, `Book an appointment`, `About`, `FAQ`, `Instagram` and the two column headings all render correctly today and keep their current text in both locales.
- The header nav and mobile panel, which read the same keys.

Do not fix this with a runtime fallback such as `?? key` or `?? ''`. A fallback paints over the next occurrence instead of failing the build, which is the same mistake as an English fallback for a missing Spanish string.

Build steps

- ☒ **1. Type the keys so the compiler checks them, then fix the labels**

Make `NavItem.key` a union derived from the dictionary rather than `string`, and remove the casts in `SiteFooter.tsx`, `SiteHeader.tsx` and `MobileNav.tsx`. Resolve the footer's service labels from the nav entries so the four service words exist once.

Expect the compiler to reject the current `siteConfig` until the keys line up. That rejection is the point of the change.

Done when: `npm run lint` and `npm run build` exit 0 with **no `as keyof` left in components/**; the built HTML for both locales has **zero** anchors with empty text inside the footer; and changing one `siteConfig` key to a name that is not in the dictionary fails the build with a type error naming it. Restore the key afterwards.

Verify

On the deployed site, both `/` and `/es`:

1. The footer Services column reads Nails, Lashes, Makeup, The gallery - in Spanish on `/es`. 2. Get in touch still reads Book an appointment, About, FAQ. Instagram is still absent, because the handle is still unknown. 3. The header nav and the mobile panel still show all five links with text.

Regression worth one look: this touches the same dictionary lookups the header and panel use, so confirm the header nav did not lose its labels while the footer gained them.

Verification performed

`npm run lint` exit 0, `npm run build` exit 0.

| Check | Result | |---|---| | `as keyof` in `components/` | **zero** | | Footer links, `en` | 7 links, **0 empty**: Nails, Lashes, Makeup, The gallery, Book an appointment, About, FAQ | | Footer links, `es` | 7 links, **0 empty**: Uñas, Pestañas, Maquillaje, La galería, Reservar una cita, Sobre mí, Preguntas frecuentes | | Header nav, both locales | 14 links, 0 empty - unchanged | | Spanish in client chunks | none. The type-only import kept both dictionaries out of the bundle |

The guard fires. Changing `nav` key `lashes` to `lashez` failed the build with three errors, including at the two lookups that previously cast the problem away:

```
data/siteConfig.ts(84,7): error TS2820: Type '"lashez"' is not assignable to
  type '"nails" | "lashes" | "makeup" | "gallery" | "about"'
components/shell/SiteHeader.tsx(50,22): error TS2551: Property 'lashez' does
  not exist ... Did you mean 'lashes'?
components/shell/MobileNav.tsx(124,22): error TS2551: ...
```

Key restored, tree clean.

Departure from the spec

The spec said to reuse the nav labels rather than duplicate four words into `footer`. That was written without noticing `nav.gallery` is "The work" and `footer.gallery` is "The gallery" - deliberately different wording. Reusing nav labels would have silently changed the footer, which the spec's own "must not break" list forbids.

So the footer keeps its own labels, and `footer` split into `headings` and `links`. That split is what makes `FooterKey` mean "a label a footer link can use": flattened, a link could have been typed with a column heading's key and still compiled. The user-visible result is identical either way, which is why this was decided here rather than sent back.

Not observed: the rendered footer on a real build. Everything above is from the built HTML and the compiler. The deployed check comes after merge.

Implementation walkthrough

`data/siteConfig.ts` - where the type now comes from

`NavItem.key` was `string`. It is now one of three unions derived from the dictionary:

```
import type { Dictionary } from "@dictionaries/en";

export type NavKey    = keyof Dictionary["nav"];
export type CtaKey    = keyof Dictionary["cta"];
export type FooterKey = keyof Dictionary["footer"]["links"];
```

Three unions rather than one, because the three lookups read three different objects. A single `LabelKey` would accept a footer key in a nav position and compile.

The import is `import type`, deliberately. Client components import `siteConfig`, and a value import of the dictionary here would have shipped both languages to every visitor - undoing the prop-passing that feature 2 set up specifically to avoid that. A type import is erased at compile time. Confirmed afterwards: no Spanish in any client chunk.

`dictionaries/` - headings split from links

`footer` became `{ headings, links }`.

This looks like tidying and is not. Flat, `FooterKey` would have included `services` and `contact`, which are column *headings*. A footer link could then be typed with a heading's key and compile fine while rendering "Get in touch" where a page name belongs. Split, `FooterKey` means exactly "a label a footer link can use".

`nails`, `lashes` and `makeup` were added to `footer.links` in both locales.

`components/shell/` - the casts removed

Three lookups, all previously cast:

```
| File | Was | Now | |---|---|---| | SiteHeader.tsx:50 | t.nav[item.key as keyof ...] |
| t.nav[item.key] | | MobileNav.tsx:124 | t.nav[item.key as keyof ...] | t.nav[item.key] | |
| SiteFooter.tsx:31 | a label() helper taking string | t.footer.links[item.key] |
```

The `label()` helper in the footer is gone entirely. It existed only to hold the cast.

The decision that went against the spec

The spec said to reuse the nav labels rather than duplicate four words into `footer`, on the reasoning that identical words in two places drift apart.

That reasoning was wrong here, and the file said so: `nav.gallery` is "The work" and `footer.gallery` is "The gallery". The footer words things differently on purpose. Reusing nav labels would have quietly changed the footer's wording, which the spec's own "must not break" list forbids two paragraphs later.

So the footer keeps its own label space. The four service words now appear in both `nav` and `footer.links` - a real duplication, accepted because the two surfaces are already worded differently and forcing them to share would need a per-item exception, which is worse than the duplication.

How it was proved

Not by reading the diff. By breaking it: `lashes` renamed to `lashez` in `siteConfig`, and the build failed in three places -

```
data/siteConfig.ts(84,7):      TS2820: '"lashez"' is not assignable to  
                                '"nails" | "lashes" | "makeup" | "gallery" | "about"'  
SiteHeader.tsx(50,22):        TS2551: Property 'lashez' does not exist  
MobileNav.tsx(124,22):        TS2551: Property 'lashez' does not exist
```

The two component errors are the point. Those exact lines were silent before.

Then the built HTML for both locales: seven footer links each, zero with empty text, and the header nav unchanged at fourteen links with none empty.

The lesson worth keeping

Feature 2 built a careful build-time guarantee that a missing translation fails the build. Three files later a convenience cast walked straight around it, and the result shipped to production.

A cast is not a type annotation. `as keyof T` does not check that a string is a key of `T` - it asserts it, and asserting is the opposite of checking. Any `as` in a lookup is worth reading as "here is where the compiler was asked to stop helping".